



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Andrews, Kurt Kent C.
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	Oct. 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
 - A laptop or desktop computer
 - Stable internet connection (for research, documentation, submission)
2. Software
 - A programming environment:
 - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
 - Any text editor (Notepad++, Sublime, VS Code) if needed
3. File Formats & Tools
 - Word processor for documentation:
 - MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF-THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

RBR was easier to implement because it only needed clear IF-THEN rules. In the code, the RBR part checks facts like scope, task_type, difficulty, and deadline_days, then gives a direct answer such as using Gemini and choosing the AI level. CBR is a little harder because it needs a case base, a similarity score, a reuse step, a revise step, and a retain step. However, CBR is more flexible for unstructured problems because it can compare a new problem with past examples even if the match is not perfect. For continuously changing data, CBR can be better because it can keep adding new cases and improve over time, while RBR needs the programmer to update the rules manually. RBR is useful in systems with fixed

rules, such as tax calculators, troubleshooting guides, or medical symptom checkers. CBR is useful in recommendation systems, customer support, legal case search, and diagnosis systems that learn from previous cases. In this program, RBR explains the matched rules, while CBR shows the similarity percentage of each past case.

Your Code - Laboratory Tasks

```
Initial Facts:
scope: School
user_type: Student
task_type: Essay Writing
difficulty: Medium
budget: Free
deadline_days: 2
needs_sources: True

Rule-Based Reasoning Matched Steps:
- Because this is free schoolwork for a student, use Gemini.
- Because the task difficulty, source requirement, and deadline need about 70 minutes.
- Because the difficulty is Medium, needs_sources is True, and deadline_days is 2, AI level is High.

Rule-Based Reasoning Solution:
Estimated minutes: 70
Type of AI to use: Gemini
Level of AI: High

Case-Based Reasoning Similarity Scores:
Case 1 similarity: 100.0%
Case 2 similarity: 57.14%
Case 3 similarity: 28.57%
Case 4 similarity: 0.0%
Case 5 similarity: 28.57%

Suggested CBR Solution:
Estimated minutes: 70
Type of AI to use: Gemini
Level of AI: High
Revise estimated minutes or press Enter to keep it unchanged: 
```

Suggested CBR Solution:
Estimated minutes: 70
Type of AI to use: Gemini
Level of AI: High
Revise estimated minutes or press Enter to keep it unchanged:
Revise type of AI or press Enter to keep it unchanged: Claude
Revise AI level (Low, Medium, High) or press Enter to keep it unchanged: Low

Case-Based Reasoning Result:
Most similar case score: 7 out of 7
Final solution:
Estimated minutes: 70
Type of AI to use: Claude
Level of AI: Low
Total cases after retain step: 6

```
def initFacts():
    facts = {
        # Category 1: Context
        "scope": "School",          # Fact 1: Environment/setting
        "user_type": "Student",     # Fact 2: User profile

        # Category 2: Task Characteristics
        "task_type": "Essay Writing", # Fact 3: What needs to be done
        "difficulty": "Medium",      # Fact 4: Task complexity level

        # Category 3: Requirements
        "budget": "Free",           # Fact 5: Cost constraint
        "deadline_days": 2,
        "needs_sources": True,
    }
    return facts

def display_facts(facts):
    print("Initial Facts:")
    for key, value in facts.items():
        print(key + ":", value)

# PART 2: Rule-Based Reasoning
def rule_schoolwork_ai(facts):
    school_tasks = ["Essay Writing", "Group Report", "Presentation"]

    if (
        facts["scope"] == "School"
        and facts["user_type"] == "Student"
        and facts["budget"] == "Free"
        and facts["task_type"] in school_tasks
    ):
        return {
            "field": "ai_type",
            "value": "Gemini",
        }
```

```

        "reason": "Because this is free schoolwork for a student, use Gemini.",
    }

def rule_professional_software_ai(facts):
    professional_tasks = ["Software Development", "Professional Work"]

    if facts["task_type"] in professional_tasks:
        return {
            "field": "ai_type",
            "value": "ChatGPT/Codex",
            "reason": "Because this is professional or software work, use ChatGPT/Codex.",
        }

def rule_web_personal_coding_ai(facts):
    if facts["task_type"] == "Personal Web Coding":
        return {
            "field": "ai_type",
            "value": "Claude",
            "reason": "Because this is web or personal coding, use Claude.",
        }

def rule_estimated_minutes(facts):
    estimated_minutes = 30

    if facts["difficulty"] == "Medium":
        estimated_minutes += 15
    elif facts["difficulty"] in ["Hard", "High"]:
        estimated_minutes += 30

    if facts["needs_sources"]:
        estimated_minutes += 15

    if facts["deadline_days"] <= 2:
        estimated_minutes += 10

    return {
        "field": "estimated_minutes",
        "value": estimated_minutes,
        "reason": "Because the task difficulty, source requirement, and deadline need about "
        + str(estimated_minutes)
        + " minutes.",
    }

def rule_ai_level(facts):
    level_score = 1

    if facts["difficulty"] == "Medium":
        level_score += 1
    elif facts["difficulty"] in ["Hard", "High"]:
        level_score += 2

    if facts["needs_sources"]:
        level_score += 1

```

```

    if facts["deadline_days"] <= 2:
        level_score += 1

    if level_score <= 1:
        ai_level = "Low"
    elif level_score <= 3:
        ai_level = "Medium"
    else:
        ai_level = "High"

    return {
        "field": "ai_level",
        "value": ai_level,
        "reason": "Because the difficulty is "
        + facts["difficulty"]
        + ", needs_sources is "
        + str(facts["needs_sources"])
        + ", and deadline_days is "
        + str(facts["deadline_days"])
        + ", AI level is "
        + ai_level
        + "."
    }

def inference_engine(facts, rules):
    solution = {}
    matched_steps = []

    for rule in rules:
        result = rule(facts)
        if result is not None:
            solution[result["field"]] = result["value"]
            matched_steps.append(result["reason"])

    return solution, matched_steps

def RBR():
    facts = initFacts()
    rules = [
        rule_schoolwork_ai,
        rule_professional_software_ai,
        rule_web_personal_coding_ai,
        rule_estimated_minutes,
        rule_ai_level,
    ]
    solution, matched_steps = inference_engine(facts, rules)

    print("Rule-Based Reasoning Matched Steps:")
    for step in matched_steps:
        print("-", step)

    print("\nRule-Based Reasoning Solution:")
    display_solution(solution)

# PART 3: Case-Based Reasoning

```



```

def initCaseBase():
    case_base = [
        {
            "problem": {
                "scope": "School",
                "user_type": "Student",
                "task_type": "Essay Writing",
                "difficulty": "Medium",
                "budget": "Free",
                "deadline_days": 2,
                "needs_sources": True,
            },
            "solution": {
                "estimated_minutes": 70,
                "ai_type": "Gemini",
                "ai_level": "High",
            },
        },
        {
            "problem": {
                "scope": "School",
                "user_type": "Student",
                "task_type": "Group Report",
                "difficulty": "Hard",
                "budget": "Free",
                "deadline_days": 5,
                "needs_sources": True,
            },
            "solution": {
                "estimated_minutes": 90,
                "ai_type": "Gemini",
                "ai_level": "High",
            },
        },
        {
            "problem": {
                "scope": "School",
                "user_type": "Student",
                "task_type": "Presentation",
                "difficulty": "Easy",
                "budget": "Low",
                "deadline_days": 3,
                "needs_sources": False,
            },
            "solution": {
                "estimated_minutes": 30,
                "ai_type": "Gemini",
                "ai_level": "Low",
            },
        },
        {
            "problem": {
                "scope": "Professional",
                "user_type": "Software Developer",
                "task_type": "Software Development",
                "difficulty": "Hard",
                "budget": "Paid",
                "deadline_days": 7,
            },
        },
    ]

```

```

        "needs_sources": False,
    },
    "solution": {
        "estimated_minutes": 120,
        "ai_type": "ChatGPT/Codex",
        "ai_level": "High",
    },
},
{
    "problem": {
        "scope": "Personal",
        "user_type": "Coder",
        "task_type": "Personal Web Coding",
        "difficulty": "Medium",
        "budget": "Free",
        "deadline_days": 4,
        "needs_sources": False,
    },
    "solution": {
        "estimated_minutes": 60,
        "ai_type": "Claude",
        "ai_level": "Medium",
    },
},
]
return case_base

def similarity_score(new_problem, old_problem):
    score = 0

    for feature, value in new_problem.items():
        if feature in old_problem and old_problem[feature] == value:
            score += 1

    return score

def retrieve_most_similar_case(new_problem, case_base):
    best_case = case_base[0]
    best_score = similarity_score(new_problem, best_case["problem"])
    case_scores = []

    for case in case_base:
        score = similarity_score(new_problem, case["problem"])
        similarity_percent = (score / len(new_problem)) * 100
        case_scores.append(similarity_percent)

        if score > best_score:
            best_case = case
            best_score = score

    return best_case, best_score, case_scores

def display_solution(solution):
    print("Estimated minutes:", solution["estimated_minutes"])
    print("Type of AI to use:", solution["ai_type"])

```

```

print("Level of AI:", solution["ai_level"])

def revise_solution(solution):
    print("\nSuggested CBR Solution:")
    display_solution(solution)

    try:
        revised_minutes = input(
            "Revise estimated minutes or press Enter to keep it unchanged: "
        ).strip()
        revised_ai_type = input(
            "Revise type of AI or press Enter to keep it unchanged: "
        ).strip()
        revised_ai_level = input(
            "Revise AI level (Low, Medium, High) or press Enter to keep it unchanged: "
        ).strip()
    except EOFError:
        revised_minutes = ""
        revised_ai_type = ""
        revised_ai_level = ""

    revised_solution = solution.copy()

    if revised_minutes:
        try:
            revised_solution["estimated_minutes"] = int(revised_minutes)
        except ValueError:
            print("Invalid number entered. Keeping estimated minutes unchanged.")

    if revised_ai_type:
        revised_solution["ai_type"] = revised_ai_type

    if revised_ai_level in ["Low", "Medium", "High"]:
        revised_solution["ai_level"] = revised_ai_level
    elif revised_ai_level:
        print("Invalid AI level entered. Keeping AI level unchanged.")

    return revised_solution

def retain_case(case_base, problem, solution):
    new_case = {
        "problem": problem,
        "solution": solution,
    }
    case_base.append(new_case)
    return case_base

def CBR():
    facts = initFacts()
    case_base = initCaseBase()

    new_problem = {
        "scope": facts["scope"],
        "user_type": facts["user_type"],
        "task_type": facts["task_type"],
    }

```

```

        "difficulty": facts["difficulty"],
        "budget": facts["budget"],
        "deadline_days": facts["deadline_days"],
        "needs_sources": facts["needs_sources"],
    }

    matched_case, score, case_scores = retrieve_most_similar_case(new_problem, case_base)

    print("\nCase-Based Reasoning Similarity Scores:")
    for index, similarity_percent in enumerate(case_scores):
        print("Case", index + 1, "similarity:", str(round(similarity_percent, 2)) + "%")

    final_solution = revise_solution(matched_case["solution"])
    retain_case(case_base, new_problem, final_solution)

    print("\nCase-Based Reasoning Result:")
    print("Most similar case score:", score, "out of", len(new_problem))
    print("Final solution:")
    display_solution(final_solution)
    print("Total cases after retain step:", len(case_base))

def main():
    facts = initFacts()
    display_facts(facts)
    print()

    RBR()
    CBR()

if __name__ == "__main__":
    main()

```